Open in app ↗

Medium

Get unlimited access to the best of Medium for less than $1/week. Become a member

Orchestrating the future: A deep dive into langchain application lifecycle

6 min read · 1 hour ago

Mrudula Pore

Listen Share ••• More

The leap from interacting with Large Language Model via chat interface to building a production -grade ,context aware application is journey through complex landscape of orchestration.

In the early days of GENERATIVE AI , developers struggled with “brittleness” of raw API calls handling state, managing long-form of memory and forcing structured output were manual , error-prone tasks

LangChain has evolved a lot from a simple wrapper to industry standard framework for building LLM-powered applications.

1. **Introduction to LangChain**

LangChain is an open-source framework that simplifies building applications using large language models. It helps developers connect LLMs with external data, tools and workflows and is available in both Python and JavaScript

IMPORTANCE IN MODERN LLM ECOSYSTEM : LangChain is a framework that simplifies the process of building applications using Large Language Models (LLMs). It provides a set of modular building blocks that developers can use to

design, integrate and deploy intelligent systems. The architecture of LangChain is structured in a way that separates the core logic, integration layers and advanced deployment tools. This layered approach makes LangChain both flexible for experimentation and robust for production use.

PROBLEM IT SOLVES :

Repurpose language models : With LangChain, organizations can repurpose LLMs for domain-specific applications without retraining or fine-tuning. Development teams can build complex applications referencing proprietary information to augment model responses. For example, you can use LangChain to build applications that read data from stored internal documents and summarize them into conversational responses. You can create a Retrieval Augmented Generation (RAG) workflow that introduces new information to the language model during prompting. Implementing context-aware workflows like RAG reduces model hallucination and improves response accuracy.

Simplify AI development : LangChain simplifies AI development by abstracting the complexity of data source integrations and prompt refining. Developers can customize sequences to build complex applications quickly. Instead of programming business logic, software teams can modify templates and libraries that LangChain provides to reduce development time.

2. **Core components of langchain : below are components of langchain**

1. LLMs
2. Prompts
3. Chains
4. Memory
5. Agents
6. Tools
7. Vector stores

LLMs : LangChain provides APIs with which developers can connect and query LLMs from their code. Developers can interface with public and proprietary models

like GPT, Bard, and PaLM with LangChain by making simple API calls instead of writing complex code

Prompts : Helps design and manage prompts using templates, making it easier to control input, output format and model behavior

Chains : Define a sequence of steps where each step can use an LLM, process data or call tools. Simple chains use one step, while multi step chains combine multiple actions.

Memory : ome conversational language model applications refine their responses with information recalled from past interactions. LangChain allows developers to include memory capabilities in their systems. It supports :

- Simple memory systems that recall the most recent conversations.
- Complex memory structures that analyze historical messages to return the most relevant results.

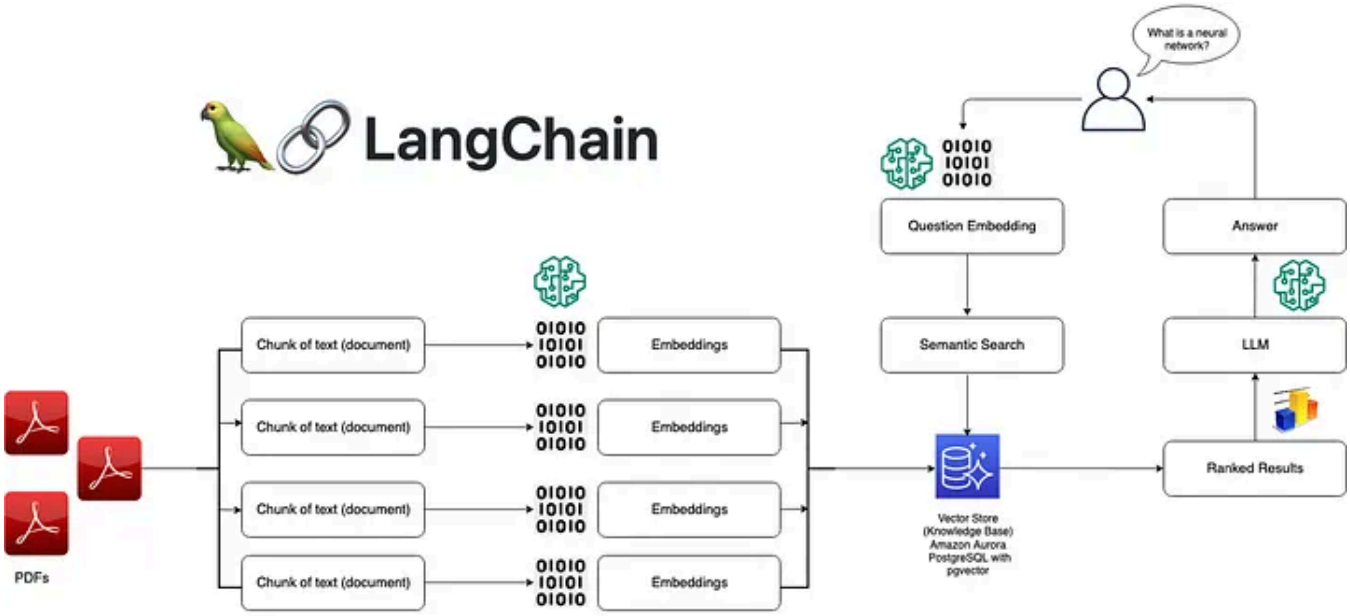
Agents : Autonomous systems that decide actions based on input, such as calling APIs or querying databases, using LLMs for decision making.An *agent* is a special chain that prompts the language model to decide the best sequence in response to a query. When using an agent, developers provide the user's input, available tools, and possible intermediate steps to achieve the desired results. Then, the language model returns a viable sequence of actions the application can take.

Tools : Tools are modular interfaces that allow LLMs to interact with real world . while LLM is naturally limited to training data , tools enable it to fetch live information, execute code

#core components of tool

1. Name : unique identifier for tool
2. Description : natural language description of what tool does
3. JSON schema : defines required inputs for the tool

Vector stores : stores data as vectors to enable similarity search, helping retrieve relevant information for tasks like document search and RAG



Langchain workflow

3. **Architecture**

LangChain enables RAG (Retrieval augmented generation) by combining document processing, vector storage and LLMs to generate accurate, context aware responses. It connects embeddings, vector databases and models into a smooth workflow

In above we can see the workflow in pictorial format

1. Document Processing

Documents (e.g., PDFs) are split into smaller chunks so they can be processed efficiently.

2. Embeddings Creation

Each chunk is converted into embeddings that capture its semantic meaning.

3. Vector Store

These embeddings are stored in a vector database, creating a searchable knowledge base.

4. User Query

The process starts when a user submits a question or request as input

5. Vector Representation

Once the query is received, LangChain converts it into a vectore representation using embeddings. This vector captures the semantic meaning of the query.

6. Similarity Search

This vector is compared with vectors stored in a database to find the most relevant matches based on meaning.

7. Fetching Relevant Information

The system retrieves the most relevant data or context from the vector database to support the response.

8. Generating a Response

The retrieved context is passed to a language model, which processes it and generates a meaningful answer.

4. CODE IMPLEMENTATION

Install dependecies

```
!pip install langchain langchain-openai python-dotenv
```

Import libraries

```
import os
from dotenv import load_dotenv
from langchain_openai import OpenAI
from langchain.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

Load API key

OPENAI_API_KEY = your_openai_api_key_here

Initialize Open AI LLM

```
llm = OpenAI(
temperature=0.7,
openai_api_key=api_key
)
```

Run simple prompt

```
prompt = "Suggest me a skill that is in demand?"
response = llm.invoke(prompt)
print(" Suggested Skill:\n", response)
```

Create prompt template

```
template = "Give me 3 career skills that are in high demand in {year}."
prompt_template = PromptTemplate.from_template(template)
```

Build a chain

LCEL (LangChain Expression Language): It's a new way to compose LLM workflows using a simple, chainable syntax with the | (pipe) operator.

1. prompt_template

- Fills placeholders (like {year}) with actual inputs.
- Example: "Give me 3 career skills in 2025."

2. llm

- Sends the formatted prompt to the OpenAI model.
- Example input: "Give me 3 career skills in 2025."
- Example output: "1. Data Analytics\n2. AI/ML\n3. Cybersecurity"

3. StrOutputParser()

- Cleans up and ensures the LLM's response is returned as a string

chain = prompt_template | llm | StrOutputParser()

RUN the chain

response = chain.invoke({"year": "2026"})
print("\n Career Skills in 2026:\n", response)

OUTPUT :

career skills in 2026

1. Data analysis and AI : with the increasing technology use and demand , professional who analyze and interpret large amount of data continue to rise in 2026
2. Digital marketing :the need of professional who can effectively market the products and services through digital channel will be in high demand
3. cybersecurity :with the rise in cyber threat and data breaches demand for cyber professionals continue to grow

5. Real use cases

 # Q &A over private documents and databases

Retrieval-Augmented Generation (RAG) is one of the most important LangChain usecases, enabling LLMs to answer questions based on your own data. Instead of relying solely on model training, LangChain allows you to embed internal documents and use a retriever to provide relevant context in real time.

This usecase is ideal for companies building internal knowledge bots, legal assistants, and customer support chat interfaces. Whether you're loading PDFs, indexing Notion pages, or querying CRM systems, LangChain provides a structured way to retrieve, chunk, and summarize information.

#Code generation and debugging assistance

LangChain excels at chaining tasks and tools together, making it a strong choice for building AI-powered engineering tools. These assistants can:

- Take user instructions and generate code snippets
- Review and explain the code line by line
- Run unit tests and highlight failing sections
- Suggest optimizations based on best practices

With LangChain, developers can integrate LLM prompts, local test environments, documentation lookups, and formatters into one pipeline. For example, a prompt might first ask the LLM to write code, then use a PythonREPLTool to validate its syntax, and finally ask the LLM to summarize the output.

These LangChain usecases improve developer productivity and reduce onboarding time for junior engineers.

6. Advantages & disadvantages

ADVANTAGE :

1. Modularity and reusability:
2. Rapid prototyping : provides out of the box implementations for common patterns
3. Massive ecosystem integrations

DISADVANTAGE:

1. Debugging complexity : Because LangChain adds multiple layers of abstraction, it can be a “black box.” When a chain fails, it can be difficult to tell if the issue is the prompt, the document retrieval, the parser, or the LLM itself.
2. Latency and overhead : Each layer of abstraction adds a small amount of processing time. For high-frequency, real-time applications where every millisecond counts, the “LangChain tax” might be too high compared to a direct API call.
3. Cost management : LangChain’s **Agents** work by looping — asking the LLM questions until a task is done. If an agent gets stuck in a logic loop, it can rack up expensive token costs very quickly without achieving a result.

WHEN TO NOT USE LANGCHAIN ?

Simple API calls : If you just need a single prompt-to-response interaction, a direct call to the OpenAI or Anthropic SDK is cleaner and faster.

Basics learning : If you are new to LLMs, using LangChain can hide how the technology actually works. It's often better to build your first RAG system from scratch to understand the mechanics.

M

Edit profile

Written by Mrudula Pore

2 followers · 3 following

Data and Ai enthusiast

No responses yet

M Mrudula Pore

What are your thoughts?

More from Mrudula Pore

Mrudula Pore

NLP PIPELINE WORKFLOW

NLP Is Referred As Natural Language Processing , It Is Subfield Of AI and CS That Uses Machine Learning To Enable Computers To Understand...

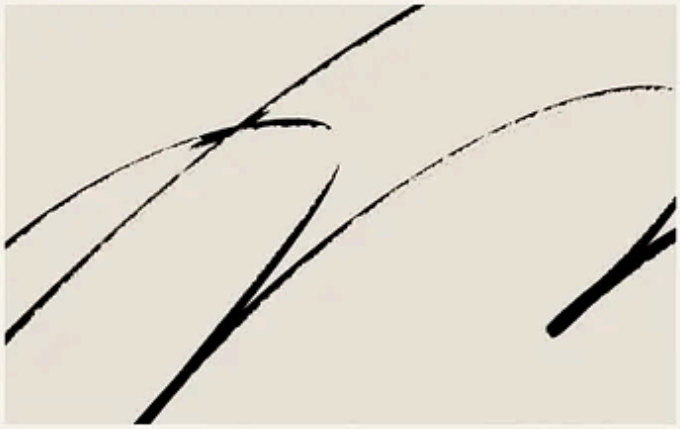
Mar 30

Mrudula Pore

HOW PYTHON USES DATA STRUCTURES BEHIND THE SCENES

“Why is python everywhere?”

Feb 14 2



Mrudula Pore

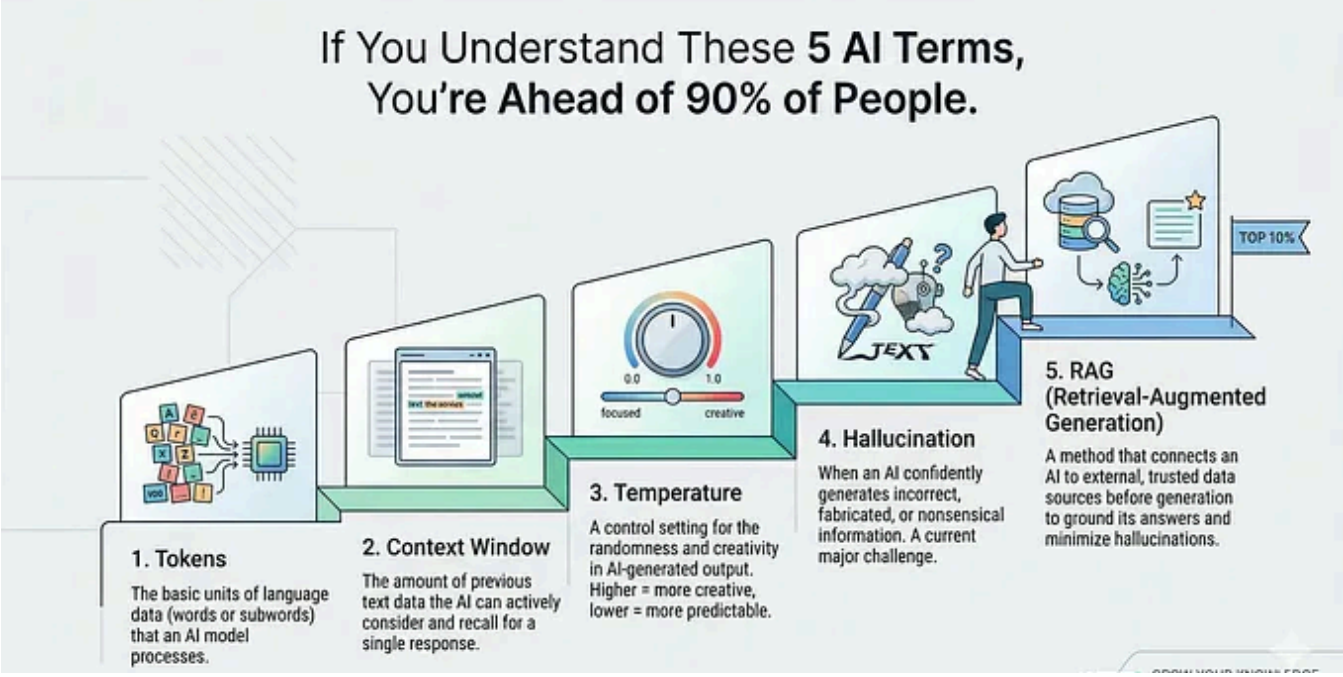
NLP TOKEN CLASSIFICATION TASKS

In our previous beginner-friendly guide to NLP, we explored how machines start understanding human language -breaking sentences into...

Apr 6

See all from Mrudula Pore

Recommended from Medium

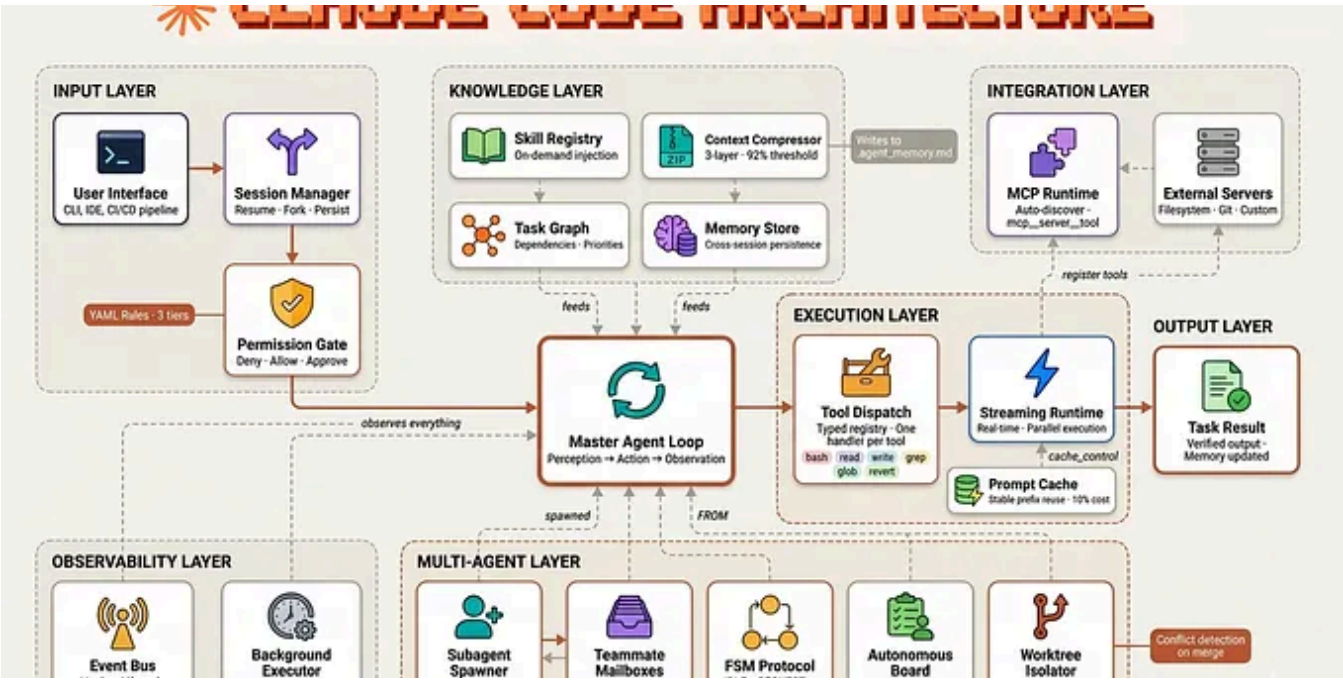


In Towards AI by Shreyas Naphad

If You Understand These 5 AI Terms, You're Ahead of 90% of People

Master the core ideas behind AI without getting lost

Mar 29 9.3K 184

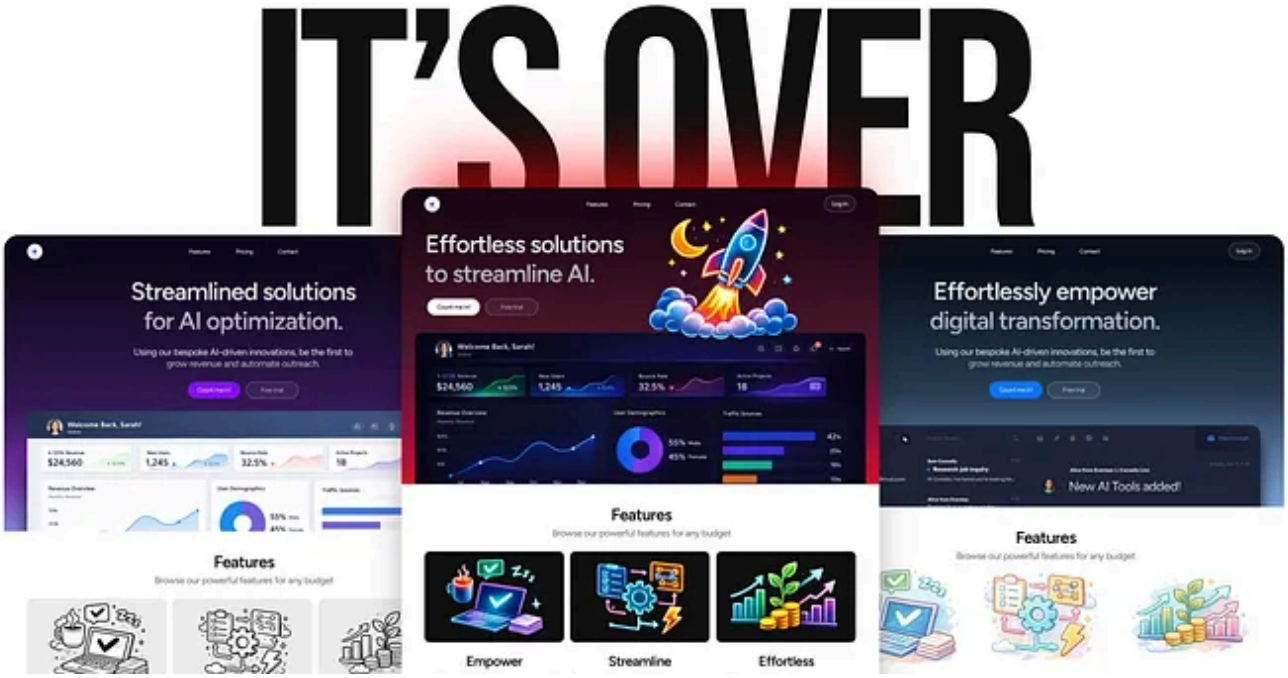


In Level Up Coding by Fareed Khan

Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

6d ago 1K 8

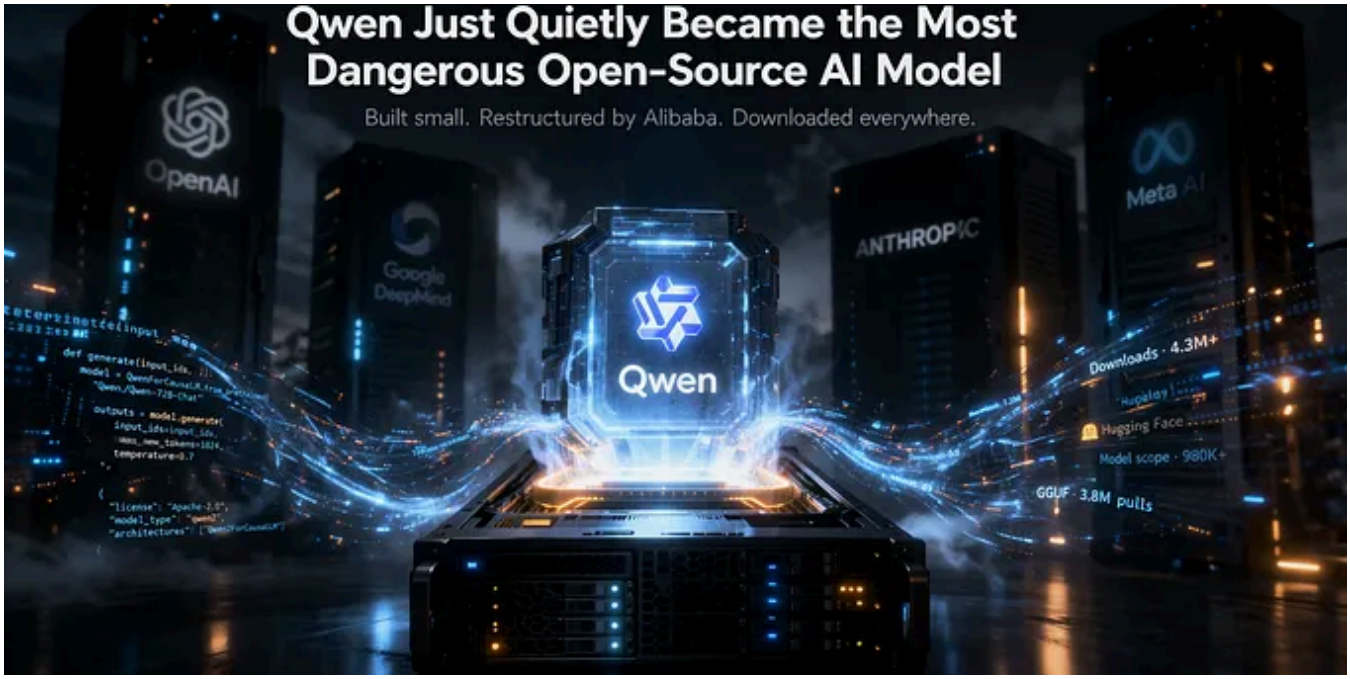


Michal Malewicz

Vibe Coding is OVER.

Here's What Comes Next.

Mar 25 5K 191



Suleiman Tawil

Qwen Just Quietly Became the Most Dangerous Open-Source AI Model

The most-downloaded AI model family on Earth was built by a small team with fewer resources than its competitors. Then Alibaba restructured...

Mar 31 1.7K 49

In Data Science Collective by Han HELOIR YAN, Ph.D. ☕

Everyone Analyzed Claude Code's Features. Nobody Analyzed Its Architecture.

Five hundred thousand lines of leaked source code reveal that the moat in AI coding tools is not the model. It is the harness.

Mar 31 4.8K 12

In ILLUMINATION by Sufyan Maan, M.Eng

I Woke Up at 4:30 AM Every Day for 30 Days — Here Is What Nobody Tells You

Here is what actually happened, from someone who did it & tracked everything.

Apr 4 7.6K 329

See more recommendations